

Day 4: API Integration & Hotel Data Fetching

Objectives

By the end of this session, you should be able to:

- Explain **frontend vs backend** and why we split them into folders
- Run a **minimal FastAPI** server with **GET** and **POST** (print-only — no database yet)
- Understand **callbacks, Promises, and async/await**
- Use **fetch()** to load hotel data and submit bookings from your Day 3 UI
- Handle **loading states and errors** in the browser

Note: Today's FastAPI is a **preview**. Week 3 goes deep on FastAPI, Pydantic, databases, and real persistence. The backend only **print()**s bookings to the terminal for now.

How to follow this lesson

You will run **two servers** at the same time:

Terminal	Folder	Command	URL
1	backend/	uvicorn main:app -- reload	http://127.0.0.1:8000
2	frontend/	npm run dev:css	Open index.html in browser

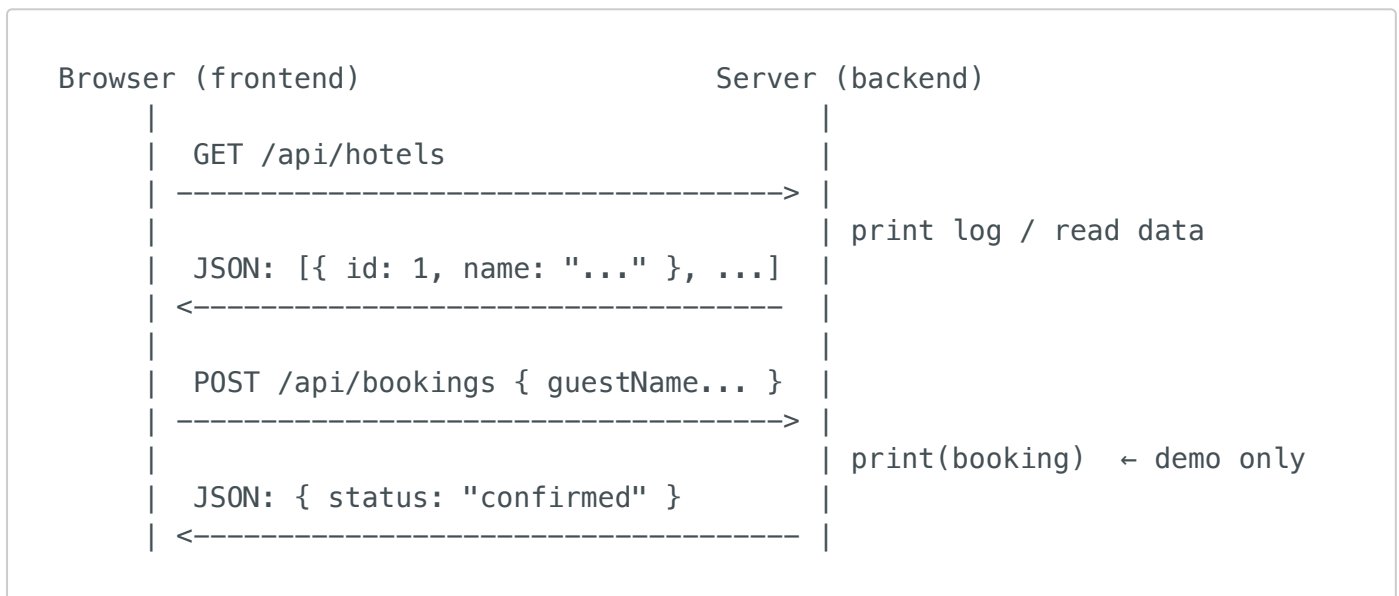
Rule: After each frontend step → save → refresh the browser.
Watch **Terminal 1** when you submit a booking — you should see a **print()** in Python.

Where this fits

Day	Focus
3	DOM + Tailwind UI (hard-coded hotels array)
4 (today)	Split frontend/backend + fetch()
5	ES modules + localStorage capstone
Week 3	FastAPI in depth (CRUD, MySQL, auth)

What is an API?

Your **frontend** (HTML + JS in the browser) talks to a **backend** (Python server) over HTTP.



- **GET** — read data (hotel list)
- **POST** — send data (booking form)

Part 1 — Project architecture

Step 1: Create the root folder

From your course projects directory:

```
mkdir hotel-booking-app
cd hotel-booking-app
```

Step 2: Folder layout

We split **frontend** and **backend** from day one — the same layout you will use in the full-stack project.

```
hotel-booking-app/
├── frontend/           # Day 3 UI (HTML, CSS, JS) – runs in the browser
│   ├── index.html
│   ├── input.css
│   ├── styles.css
│   ├── app.js
│   └── package.json
└── backend/           # FastAPI – runs on your machine (Python)
    ├── main.py
    └── requirements.txt
```

Checkpoint: You have an empty `hotel-booking-app` folder.

Step 3: Copy your Day 3 frontend

Copy your working `hotel-booking-ui` project into `frontend/`:

```
# Example: if Day 3 was next to this folder
cp -r ../hotel-booking-ui ./frontend
```

Or copy these files manually into `frontend/`:

- `index.html`, `input.css`, `app.js`, `package.json`
- Run `npm install` and `npm run build:css` inside `frontend/`

Checkpoint: `frontend/index.html` opens in the browser and still shows hotel cards (from the hard-coded array).

Step 4: Create the backend folder

```
mkdir backend
cd backend
```

Checkpoint: Structure matches the diagram above.

Part 2 — Minimal FastAPI backend

Step 5: Python virtual environment

```
cd backend
python3 -m venv .venv
source .venv/bin/activate    # Windows: .venv\Scripts\activate
```

You should see `(.venv)` in your terminal prompt.

Step 6: Install FastAPI and Uvicorn

Create `backend/requirements.txt`:

```
fastapi
uvicorn[standard]
```

Install:

```
pip install -r requirements.txt
```

Step 7: GET /api/hotels — return hotel JSON

Create `backend/main.py`:

```
from fastapi import FastAPI

app = FastAPI(title="Hotel Booking API (Day 4 demo)")

# Hard-coded data — no database yet (Week 3 adds real storage)
HOTELS = [
    {
        "id": 1,
        "name": "Grand Palace Hotel",
        "location": "Foshan",
        "price": 200,
        "rating": 4.5,
        "available": True,
        "amenities": ["WiFi", "Pool", "Spa"],
    },
    {
        "id": 2,
        "name": "Seaside Resort",
        "location": "Guangzhou",
        "price": 150,
        "rating": 4.0,
        "available": True,
        "amenities": ["WiFi", "Beach", "Pool"],
    },
    {
        "id": 3,
        "name": "Mountain Lodge",
        "location": "Shenzhen",
        "price": 180,
        "rating": 4.8,
        "available": False,
        "amenities": ["WiFi", "Fireplace", "Hiking"],
    },
]

@app.get("/api/hotels")
def get_hotels():
    print("GET /api/hotels — sending hotel list")
    return HOTELS
```

Step 8: Start the server and test GET

```
uvicorn main:app --reload
```

Open in your browser:

- <http://127.0.0.1:8000/api/hotels> — you should see JSON
- <http://127.0.0.1:8000/docs> — auto-generated API docs (Swagger)

Terminal 1 should print: `GET /api/hotels – sending hotel list`

Checkpoint: JSON array of 3 hotels in the browser.

Step 9: `POST /api/bookings` — print only

Add to `main.py` (below the GET route):

```
from pydantic import BaseModel

class BookingRequest(BaseModel):
    guestName: str
    email: str
    hotelId: int
    hotelName: str
    checkIn: str
    checkOut: str
    total: str

@app.post("/api/bookings")
def create_booking(booking: BookingRequest):
    # Demo only – print to server terminal (no database yet)
    print("POST /api/bookings – new booking:")
    print(f"  Guest: {booking.guestName} <{booking.email}>")
    print(f"  Hotel: {booking.hotelName} (id={booking.hotelId})")
    print(f"  Dates: {booking.checkIn} → {booking.checkOut}")
    print(f"  Total: {booking.total}")

    return {
        "status": "confirmed",
        "message": f"Booking received for {booking.guestName} (demo – not saved yet)",
    }
```

Test in <http://127.0.0.1:8000/docs>:

1. Open **POST /api/bookings** → **Try it out**
2. Paste sample JSON and **Execute**
3. Check **Terminal 1** for the `print()` output

Checkpoint: Server terminal shows booking details; browser shows JSON response.

Step 10: Enable CORS (so the browser can call the API)

The frontend runs on a different “origin” than the API. Add **CORS middleware** at the top of `main.py`, right after `app = FastAPI(...)`:

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],    # OK for local dev; restrict in production
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Restart is automatic if `--reload` is on.

Checkpoint: No code visible change yet - CORS matters in Step 17 when `fetch` runs from `index.html`.

Part 3 — Async JavaScript

Before connecting the UI, understand **why** `fetch` feels different from normal code.

Step 11: Synchronous vs asynchronous

```
console.log('A');
console.log('B');
console.log('C');
// Always prints A, B, C in order
```

Network requests **take time**. We don't want to freeze the page while waiting.

```
console.log('A');
fetch('http://127.0.0.1:8000/api/hotels'); // starts request, does not wait
console.log('B');
// Often prints A, B, then data arrives later
```

Step 12: Callbacks

```
function loadHotelsCallback(onSuccess, onError) {
  fetch('http://127.0.0.1:8000/api/hotels')
    .then((response) => response.json())
    .then((data) => onSuccess(data))
    .catch((err) => onError(err));
}

loadHotelsCallback(
  (hotels) => console.log('Got hotels:', hotels),
  (err) => console.error('Failed:', err)
);
```

Nested callbacks become hard to read (“callback hell”). **Promises** help.

Step 13: Promises with `.then()`

```
fetch('http://127.0.0.1:8000/api/hotels')
  .then((response) => {
    if (!response.ok) throw new Error('Network error');
    return response.json();
  })
  .then((hotels) => {
    console.log('Hotels:', hotels);
  })
```



```
.catch((error) => {  
  console.error('Error:', error);  
});
```

Piece	Meaning
<code>fetch(url)</code>	Returns a Promise
<code>.then()</code>	Run when success
<code>.catch()</code>	Run when error
<code>response.json()</code>	Parse JSON body (also returns a Promise)

Step 14: `async` / `await` (recommended)

```
async function loadHotels() {  
  try {  
    const response = await fetch('http://127.0.0.1:8000/api/hotels');  
    if (!response.ok) throw new Error(`HTTP ${response.status}`);  
    const hotels = await response.json();  
    console.log('Hotels:', hotels);  
    return hotels;  
  } catch (error) {  
    console.error('Failed to load hotels:', error);  
    throw error;  
  }  
}
```

`async/await` is syntax sugar over Promises — easier to read, same idea.

Practice (DevTools Console): Paste the function above, call `loadHotels()` while the backend is running.

Step 15: JSON — what travels over the wire

APIs send **JSON** (text that looks like JavaScript objects).

```
const booking = { guestName: 'Li Wei', hotelId: 1 };
const body = JSON.stringify(booking);    // object → string for POST

const parsed = JSON.parse(body);        // string → object
```

For **POST**, you send a string:

```
fetch(url, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(booking),
});
```

Part 4 — Connect frontend to backend

Open `frontend/app.js`. We replace the hard-coded `hotels` array with API calls.

API base URL — add at the top of `app.js`:

```
const API_BASE = 'http://127.0.0.1:8000';
```

Step 16: Fetch hotels on page load

Remove the hard-coded `const hotels = [...]` array.

Add:

```
let hotels = [];

async function loadHotelsFromApi() {
  const response = await fetch(`${API_BASE}/api/hotels`);
  if (!response.ok) throw new Error('Failed to load hotels');
  hotels = await response.json();
}
```

Change the bottom of `app.js` from `displayHotels(hotels)` to:

```
async function init() {
  try {
    await loadHotelsFromApi();
    displayHotels(hotels);
  } catch (error) {
    console.error(error);
    alert('Could not load hotels. Is the backend running on port 8000?');
  }
}

init();
```

Before testing: Start backend (`uvicorn main:app --reload`).

Save → refresh.

You should see: Same three hotel cards, but data now comes from FastAPI.

Terminal 1: `GET /api/hotels – sending hotel list`

Step 17: Loading indicator

Add to `index.html` inside the gray wrapper, above `#hotelList`:

```
<p id="loadingMessage" class="text-center text-gray-500 mb-4 hidden">Loading
hotels...</p>
<p id="errorMessage" class="text-center text-red-600 mb-4 hidden"></p>
```

Update `init()`:

```
async function init() {
  const loading = document.getElementById('loadingMessage');
  const errorEl = document.getElementById('errorMessage');

  loading.classList.remove('hidden');
  errorEl.classList.add('hidden');

  try {
    await loadHotelsFromApi();
    displayHotels(hotels);
  } catch (error) {
    console.error(error);
    errorEl.textContent = 'Could not load hotels. Start the backend: uvicorn
```

```
main:app --reload';
  errorEl.classList.remove('hidden');
} finally {
  loading.classList.add('hidden');
}
}
```

Test: Stop the backend → refresh → you should see the error message, not a frozen page.

Step 18: Submit booking with POST

Replace the booking `submit` handler in `app.js` with:

```
document.getElementById('bookingForm').addEventListener('submit', async
(event) => {
  event.preventDefault();
  if (!selectedHotel) return;

  const booking = {
    guestName: document.getElementById('guestName').value,
    email: document.getElementById('email').value,
    hotelId: selectedHotel.id,
    hotelName: selectedHotel.name,
    checkIn: document.getElementById('checkIn').value,
    checkOut: document.getElementById('checkOut').value,
    total: document.getElementById('totalPrice').textContent,
  };

  try {
    const response = await fetch(`${API_BASE}/api/bookings`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(booking),
    });

    if (!response.ok) throw new Error('Booking failed');

    const result = await response.json();
    console.log('Server response:', result);
    alert(result.message);
    closeBookingModal();
  } catch (error) {
    console.error(error);
    alert('Could not submit booking. Is the backend running?');
  }
});
```

Test: Book a hotel → check **Terminal 1** for `print()` output → alert in browser.

Step 19: Keep client-side filtering

`filterHotels()` from Day 3 still works — it filters the `hotels` array already loaded in memory. No API change needed for search today. (Week 3 can add query parameters like `GET /api/hotels?location=Foshan.`)

Part 5 — Class activity

Task: End-to-end demo

1. Start backend and frontend CSS watcher
 2. Load page — hotels from API
 3. Search / filter in the UI
 4. Submit a booking — see `print()` in Python terminal
 5. Open <http://127.0.0.1:8000/docs> and explain GET vs POST to a partner
-

Troubleshooting

Problem	Fix
Failed to fetch / CORS error	Add CORS middleware (Step 10); restart uvicorn
Could not load hotels	Backend not running — <code>cd backend && uvicorn main:app --reload</code>
Empty page, no error	Open DevTools Console (F12)
POST works in <code>/docs</code> but not from browser	CORS missing or wrong URL — use <code>http://127.0.0.1:8000</code>
<code>pip: command not found</code>	Use <code>python3 -m pip install -r requirements.txt</code>

Problem	Fix
Hotels show but booking fails	Check JSON field names match <code>BookingRequest</code> in <code>main.py</code>

Homework

Pick **2–3**:

1. Add `GET /api/hotels/{hotel_id}` that prints and returns one hotel
2. Show a “Submitting...” state on the booking button during POST
3. Log `response.status` in the console for every `fetch`
4. Read FastAPI docs: <https://fastapi.tiangolo.com/> (preview for Week 3)

Read before Day 5:

- ES Modules: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules>
- `localStorage`: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>

Summary

Part	You learned
1	<code>frontend/</code> + <code>backend/</code> folder split
2	Minimal FastAPI: GET hotels, POST booking (<code>print</code> only)
3	Callbacks → Promises → <code>async/await</code> , JSON
4	<code>fetch()</code> from Day 3 UI, loading + error states

Week 3 preview: Real FastAPI with Pydantic validation, SQLAlchemy, MySQL, and connecting this same frontend to a production API.

Next (Day 5): Refactor into ES modules + `localStorage` for favorites and booking drafts.

Appendix: Complete working project

Folder structure

```
hotel-booking-app/  
├── frontend/  
│   ├── index.html  
│   ├── input.css  
│   ├── styles.css      # generated  
│   ├── app.js  
│   └── package.json  
└── backend/  
    ├── main.py  
    ├── requirements.txt  
    └── .venv/
```

Run commands

Terminal 1 — backend:

```
cd backend  
source .venv/bin/activate  
uvicorn main:app --reload
```

Terminal 2 — frontend CSS:

```
cd frontend  
npm run dev:css
```

Open `frontend/index.html` in the browser.

backend/requirements.txt

```
fastapi
uvicorn[standard]
```

backend/main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

app = FastAPI(title="Hotel Booking API (Day 4 demo)")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

HOTELS = [
    {
        "id": 1,
        "name": "Grand Palace Hotel",
        "location": "Foshan",
        "price": 200,
        "rating": 4.5,
        "available": True,
        "amenities": ["WiFi", "Pool", "Spa"],
    },
    {
        "id": 2,
        "name": "Seaside Resort",
        "location": "Guangzhou",
        "price": 150,
        "rating": 4.0,
        "available": True,
        "amenities": ["WiFi", "Beach", "Pool"],
    },
    {
        "id": 3,
        "name": "Mountain Lodge",
        "location": "Shenzhen",
        "price": 180,
        "rating": 4.8,
        "available": False,
        "amenities": ["WiFi", "Fireplace", "Hiking"],
    },
]
```



```

class BookingRequest(BaseModel):
    guestName: str
    email: str
    hotelId: int
    hotelName: str
    checkIn: str
    checkOut: str
    total: str

@app.get("/api/hotels")
def get_hotels():
    print("GET /api/hotels – sending hotel list")
    return HOTELS

@app.get("/api/hotels/{hotel_id}")
def get_hotel(hotel_id: int):
    print(f"GET /api/hotels/{hotel_id}")
    for hotel in HOTELS:
        if hotel["id"] == hotel_id:
            return hotel
    return {"error": "Hotel not found"}

@app.post("/api/bookings")
def create_booking(booking: BookingRequest):
    print("POST /api/bookings – new booking:")
    print(f"  Guest: {booking.guestName} <{booking.email}>")
    print(f"  Hotel: {booking.hotelName} (id={booking.hotelId})")
    print(f"  Dates: {booking.checkIn} → {booking.checkOut}")
    print(f"  Total: {booking.total}")

    return {
        "status": "confirmed",
        "message": f"Booking received for {booking.guestName} (demo – not
saved yet)",
    }

```

frontend/app.js (API-connected — key changes from Day 3)

Use your full Day 3 `app.js` and apply these changes:

1. Remove hard-coded `const hotels = [...]`
2. Add `API_BASE` and `loadHotelsFromApi`
3. Replace form submit with `fetch` POST
4. Replace bottom `displayHotels(hotels)` with `init()`

```

const API_BASE = 'http://127.0.0.1:8000';

let hotels = [];
let selectedHotel = null;

async function loadHotelsFromApi() {
  const response = await fetch(`${API_BASE}/api/hotels`);
  if (!response.ok) throw new Error('Failed to load hotels');
  hotels = await response.json();
}

function createHotelCard(hotel) {
  const card = document.createElement('article');
  card.className =
    'bg-white rounded-lg shadow-md p-6 hover:shadow-xl hover:-translate-y-1 transition-all duration-300 animate-fade-in';

  const availabilityClass = hotel.available ? 'text-green-600' : 'text-red-600';
  const availabilityText = hotel.available ? 'Available' : 'Booked';

  card.innerHTML = `
    <h3 class="text-xl font-bold text-hotel-primary mb-2">${hotel.name}</h3>
    <p class="text-gray-600 mb-2">📍 ${hotel.location}</p>
    <p class="text-2xl font-bold text-hotel-secondary mb-2">$$${hotel.price}/night</p>
    <p class="text-hotel-gold mb-3">★ ${hotel.rating} stars</p>
    <div class="flex flex-wrap gap-2 mb-3">
      ${hotel.amenities
        .map((a) => `<span class="bg-gray-100 px-2 py-1 rounded text-sm">${a}</span>`)
        .join('')}
    </div>
    <p class="${availabilityClass} mb-4">${availabilityText}</p>
    ${
      hotel.available
        ? `<button type="button" data-book-hotel="${hotel.id}"
          class="w-full bg-hotel-secondary text-white py-2 rounded
hover:bg-hotel-primary">
          Book Now
        </button>`
        : ''
    }
  `;
  return card;
}

function displayHotels(hotelsToShow) {
  const hotelList = document.getElementById('hotelList');
  hotelList.innerHTML = '';
  hotelsToShow.forEach((hotel) =>
    hotelList.appendChild(createHotelCard(hotel)));
}

function filterHotels() {
  const searchTerm =

```

```

document.getElementById('searchInput').value.toLowerCase();
const locationFilter = document.getElementById('locationFilter').value;
const maxPrice = document.getElementById('maxPrice').value;

const filtered = hotels.filter((hotel) => {
  const matchesSearch =
    hotel.name.toLowerCase().includes(searchTerm) ||
    hotel.location.toLowerCase().includes(searchTerm);
  const matchesLocation = !locationFilter || hotel.location ===
locationFilter;
  const matchesPrice = !maxPrice || hotel.price <= Number(maxPrice);
  return matchesSearch && matchesLocation && matchesPrice;
});

displayHotels(filtered);
}

function openBookingModal(hotelId) {
  selectedHotel = hotels.find((h) => h.id === hotelId) ?? null;
  document.getElementById('bookingModal').classList.remove('hidden');
}

function closeBookingModal() {
  document.getElementById('bookingModal').classList.add('hidden');
  selectedHotel = null;
  document.getElementById('bookingForm').reset();
  document.getElementById('totalNights').textContent = '0';
  document.getElementById('totalPrice').textContent = '$0';
}

function calculateTotal() {
  const checkIn = document.getElementById('checkIn').value;
  const checkOut = document.getElementById('checkOut').value;
  if (!checkIn || !checkOut || !selectedHotel) return;

  const nights = Math.ceil(
    (new Date(checkOut) - new Date(checkIn)) / (1000 * 60 * 60 * 24)
  );
  if (nights > 0) {
    document.getElementById('totalNights').textContent = String(nights);
    document.getElementById('totalPrice').textContent = `$$${nights *
selectedHotel.price}`;
  }
}

document.getElementById('searchInput').addEventListener('input',
filterHotels);
document.getElementById('locationFilter').addEventListener('change',
filterHotels);
document.getElementById('maxPrice').addEventListener('input', filterHotels);

document.getElementById('hotelList').addEventListener('click', (event) => {
  const button = event.target.closest('[data-book-hotel]');
  if (!button) return;
  openBookingModal(Number(button.dataset.bookHotel));
});

```

```

document.getElementById('closeModal').addEventListener('click',
closeBookingModal);
document.getElementById('checkIn').addEventListener('change',
calculateTotal);
document.getElementById('checkOut').addEventListener('change',
calculateTotal);

document.getElementById('bookingForm').addEventListener('submit', async
(event) => {
    event.preventDefault();
    if (!selectedHotel) return;

    const booking = {
        guestName: document.getElementById('guestName').value,
        email: document.getElementById('email').value,
        hotelId: selectedHotel.id,
        hotelName: selectedHotel.name,
        checkIn: document.getElementById('checkIn').value,
        checkOut: document.getElementById('checkOut').value,
        total: document.getElementById('totalPrice').textContent,
    };

    try {
        const response = await fetch(`${API_BASE}/api/bookings`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(booking),
        });
        if (!response.ok) throw new Error('Booking failed');
        const result = await response.json();
        alert(result.message);
        closeBookingModal();
    } catch (error) {
        console.error(error);
        alert('Could not submit booking. Is the backend running?');
    }
});

async function init() {
    const loading = document.getElementById('loadingMessage');
    const errorEl = document.getElementById('errorMessage');

    if (loading) loading.classList.remove('hidden');
    if (errorEl) errorEl.classList.add('hidden');

    try {
        await loadHotelsFromApi();
        displayHotels(hotels);
    } catch (error) {
        console.error(error);
        if (errorEl) {
            errorEl.textContent = 'Could not load hotels. Start backend: uvicorn
main:app --reload';
            errorEl.classList.remove('hidden');
        }
    } finally {
        if (loading) loading.classList.add('hidden');
    }
}

```

```
}  
}  
  
init();
```

`index.html` is unchanged from Day 3, plus optional loading/error lines from Step 17.

Lab reference: [lab_new/steps/step04_api_fetch/](#)